

PDPA HACKATHON 2026 · TEAM PDPG

We protect data in use.

Personal Data Protection Gateway — the data processor computes on data it cannot read. Encrypted in, encrypted out, only the controller holds the key.

We protect data **in use.**

Everything in this deck is live and public — scan any of these while you read.



pdpglab.xyz
docs · education · playground



github.com/PDPG-lab/pydpdp
source · tests · Thai docs



Colab — getting started
real encryption, ~2 min, editable

🇹🇭 ฉบับภาษาไทยเริ่มที่หน้า 34

Encryption today protects two of three states.

At rest



encrypted on disk — solved, standard practice

In transit



encrypted on the wire — solved, standard practice

In use



the moment anyone computes on it, it must be decrypted — somewhere that isn't yours

Sensitive data leaves home. Liability doesn't.



มาตรา 37(4) — breach? notify สคส. within 72 hours

มาตรา 83 — administrative fines up to ฿3,000,000

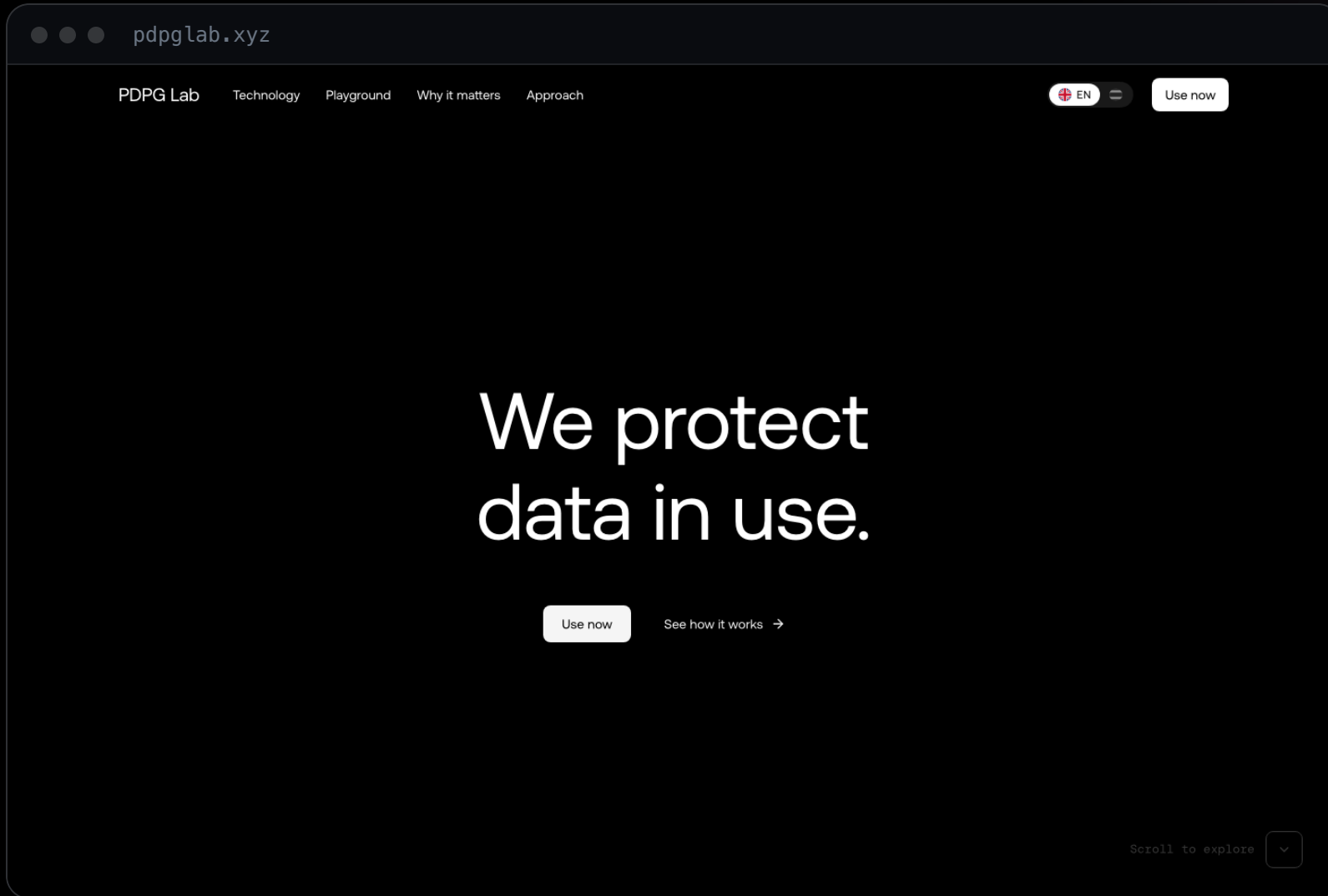
DPA (พ.40 ๖.3) — contracts punish; **they don't prevent**

Behind every record is a real person. A leak becomes scams, identity theft, broken trust — harm to the data subject and to the controller's reputation that **no fine repairs**.

What if the processor **never saw the data at all?**

PDPG: the same algorithms, the same code — running on **ciphertext**.

At rest ✓ · in transit ✓ · and now, in use ✓



- **Live now** — docs, education, interactive playground
- Built for the people who don't write code: **DPOs, legal, executives**
- Thai and English



pdpglab.xyz

Feel the idea before you trust it.

LEVEL 01

Sealed numbers

add and multiply values you
cannot see

LEVEL 02

Protected vs exposed

what each party actually
sees, side by side

LEVEL 03

Blind scoring

a credit model runs; the
values never appear

LEVEL 04

ML models

the full controller–processor
round trip

A conceptual walkthrough in the browser — **then one click to the real cryptography in Colab.**

RUN IT — OPEN SOURCE, TODAY

The screenshot shows the GitHub repository page for `PDPG-lab/pydpdp`. The repository is public and has 1 branch and 0 tags. The main branch is `main`. The repository description is "Encrypted execution for the Python data stack — numpy, pandas, scikit-learn on homomorphically encrypted data. A PDPG-lab project — pdpglab.xyz". The repository has 0 stars, 0 forks, and 0 watchers. The repository is reported by the PDPG-lab project.

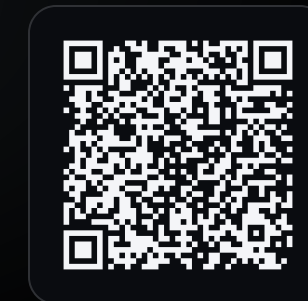
The repository files and folders are listed below:

File/Folder	Description	Time
<code>demo</code>	docs: FAFO by introspection — grab the raw ciphertext, ge...	2 hours ago
<code>docs</code>	feat: pdpg.sklearn.install() — targeted, type-gated runtime...	42 minutes ago
<code>src/pydpdp</code>	feat: pdpg.sklearn.install() — targeted, type-gated runtime...	42 minutes ago
<code>tests</code>	feat: pdpg.sklearn.install() — targeted, type-gated runtime...	42 minutes ago
<code>.gitignore</code>	chore: project skeleton (pyproject, gitignore, readme stub)	15 hours ago
<code>LICENSE.md</code>	docs: point homepage and enterprise contact at pdpglab....	5 hours ago
<code>README.md</code>	feat: pdpg.sklearn.install() — targeted, type-gated runtime...	42 minutes ago
<code>README.th.md</code>	docs: note FHE's research velocity as the rationale for bac...	2 hours ago
<code>confest.py</code>	feat: arithmetic dispatch — + - * / scalar, broadcasts, neg	15 hours ago
<code>pyproject.toml</code>	docs: CKKS/TenSEAL framed as the first backend, not the ...	2 hours ago

The repository has a README file and a License file. The README file is titled `pydpdp` and has a description: "Encrypted execution for the Python data stack — numpy, pandas, scikit-learn on homomorphically encrypted data. A PDPG-lab project — pdpglab.xyz". The License file is titled `License` and has a description: "PDPG Community".

The repository has 2 contributors: `claude` and `yoyoismee`.

- **pip install**, one line
- **111 automated tests**, all green
- **5 Colab notebooks** — run the real encryption during judging, edit it, keep it
- `numpy · pandas · scikit-learn`



Colab — getting started

Two lines per side. The rest is our problem.

DATA CONTROLLER

```
ctx = pdpg.Context.create()
pdpg.encrypt(df, ctx).save("data.enc")
# ... later: the only party who can read
result = pdpg.load("result.enc").decrypt()
```

DATA PROCESSOR

```
pdpg.activate("processor.ctx") # no secret key
pdpg.install() # np.load speaks .enc
X = np.load("data.enc")
scores = X @ w + b # unchanged code, blind
```

8 ms

score 200 applicants, encrypted

2.5×10^{-4}

max error vs plaintext

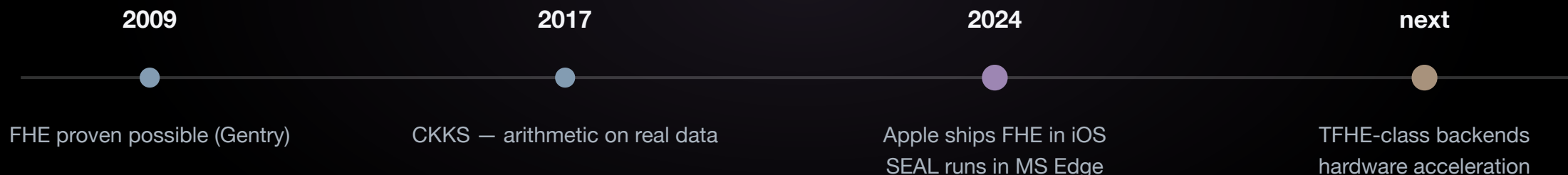
111

automated tests, green

FHE

CKKS backend · swappable by design

The math is 15 years old. It reached your pocket last year.



FHE performance is improving at a compounding rate, with dedicated silicon on the way. PDPG is backend-neutral by design — **workloads written today inherit that curve. The engine improves; your code doesn't.**

Free for the public good. Funded by those who profit most.

PDPG Community · free

- Government & public sector
- Education, research, non-profits
- Companies under **฿50M** (TH) / **\$1M** revenue

Enterprise · licensed

- Larger organizations license the same source-available stack
- Support & consulting available
- **Their license fees fund the free tier**

TEAM PDPG



**Teeramet
Kunpittaya**



**Phatrasek
Jirabovonvisut**



**Pakorn
Nathong**

CRYPTOGRAPHY

- Author, **Zama's best FHE-ECDSA solution**
- Ex-Privacy researcher @ **Ethereum Foundation**
- Co-author, ZKP paper — **100+ citations**
- Contributor, the **biggest privacy project in blockchain**

AI & STARTUP

- Co-author, **Typhoon** — the first Thai LLM
- Built an AI app with **1M+ global users**
- Ex-Senior Data Scientist @ **Sertis**
- Ex-Research Lead @ **SCB10X** Venture Builder
- Advisor @ **ExpresSo** (PTT Group VC & VB)
- Ex-Chief Intern (unofficial) @ **SCB10X**

RECOGNITION & EDUCATION

- Winner, **ZK Hack Istanbul**
- Grantee of **multiple international companies**
- **King's Scholar** · CS @ Columbia
- B.Eng @ **Chulalongkorn**
- Founder, **ETH Pad Thai** (non-profit edu)
- Author, **"Hodl My Beer"** (education)

We build systems that work on data they cannot read.

The processor never sees the data. That's the product.



pdpglab.xyz



[GitHub](#)



[Live demo \(Colab\)](#)

FOR THE READING JUDGE

Appendix

Evidence, measurements, architecture, legal mapping, honest limits.

A1 · TECHNOLOGY READINESS, LAYER BY LAYER

LAYER	READINESS	EVIDENCE
FHE / CKKS mathematics	Settled science; mass-market	Proven 2009 (Gentry); CKKS 2017; 15+ years of open review; lattice family behind NIST post-quantum standards; Apple ships FHE in iOS (2024)
Microsoft SEAL (via TenSEAL)	Production software (TRL 8–9)	Maintained by Microsoft Research; same library running in MS Edge Password Monitor
pypdpg — our innovation	Working PoC (TRL 4–5)	Open source · pip install · 111 automated tests · numpy / pandas / scikit-learn · CLI
Interactive notebooks ×5	Run-anywhere, editable	Real encryption end-to-end on a fresh Colab in ~2 min; usable as starting points
Project site + playground	Ready to use	Docs & education for non-programmers (DPO, legal, executives) — pdpglab.xyz

A2 · MEASURED PERFORMANCE — LOAN SCORING, 200 APPLICANTS × 5 FEATURES (APPLE M4 PRO)

OPERATION	MEASURED	HONEST COSTS	VALUE
Key generation (one-time)	~1.3 s	Max abs error vs plaintext	2.5×10^{-4}
Encrypt 200×5	30 ms	Ciphertext size (this demo)	5.3 MB (~660×)
Blind scoring — $X @ w + b$	8 ms	... at full slot utilisation	~16×
Column means (blind)	373 ms	Evaluation keys (one-time ship)	179.5 MB
Sigmoid approx (blind)	12 ms	Multiplicative depth budget	4 levels
Decrypt result	2 ms	Rows per array	$\leq 8,192$

Every number reproducible from the public repository.

A3 · LIBRARY COVERAGE & CAPABILITY BUCKETS

LIBRARY	STATUS	RUNS ON CIPHERTEXT
numpy	core	arithmetic, broadcasting, @ / dot, reductions, np.load drop-in — via official dispatch protocols
pandas	facade	named columns, computed columns, labeled aggregates (CipherFrame)
scikit-learn	inference hooks	linear models, logistic probabilities, scalers, KMeans distances, Pipelines — one line: pdpg.sklearn.install()
shell	CLI	pdpg keygen / encrypt / inspect / decrypt
BUCKET	MEANING	
Drop in now	existing code runs unchanged (linear algebra, statistics, fitted linear models)	
Needs a rewrite	data-dependent logic, written branchless — if/else → gate·b + (1–gate)·c — runs today	
Waiting on backends	exact compare / sort / division / unlimited depth — arrive with TFHE-class backends, code unchanged	

A4 · ERRORS THAT TEACH — BACKEND-AWARE REFUSALS

BACKEND-LIMITED (ROADMAP INCLUDED)

```
>>> X > 600
```

`np.greater (>)` is not supported by the current backend (CKKS).

Why: the current backend cannot evaluate comparisons at all — and even on a comparison-capable backend the result stays encrypted; reading the verdict needs the key. That's the point.

Do instead: return the encrypted result to the data owner, or soft-threshold with `pdpq.approx.sigmoid(x)`.

Backend roadmap: TFHE-class backends compute comparisons with encrypted results; workload code won't change.

IMPOSSIBLE BY DESIGN (ANY BACKEND)

```
>>> np.asarray(X)
```

`np.asarray(X)` — coercing a `CipherArray` to a plain `ndarray` is impossible on this side of the trust boundary.

Why: this needs the plaintext value in this process, and that value exists only where the secret key lives.

Do instead: keep computing on the `CipherArray`, or `.decrypt()` on the data owner's side.

Every refused operation explains itself — never a raw stack trace. The distinction mirrors our roadmap honestly.

Python has no private fields. It doesn't matter.

```
>>> inner = X["income"]._vectors[0]      # reach past the API – nothing stops you
>>> blob = inner.serialize()
CKKSVector: 1,052,889 bytes. 96 from the middle:

0008086c  0c 43 75 52 f9 4d 40 2d 00 cc 2f 28 6c 9f 0b c6  |.CuR.M@-../(l...|
0008087c  03 9f b2 b6 85 cf c0 db 0f 9d 44 a6 60 64 05 27  |.....D.`d.'|
0008088c  02 45 f3 92 d5 75 74 5d 04 8d 72 d2 3a 76 fe 5a  |.E...ut]..r.:v.Z|
0008089c  0a 74 04 0c c0 a5 81 d6 0b 2a f5 ba fe 41 56 dc  |.t.....*...AV.|
```

200 incomes, statistically indistinguishable from random noise — and different noise on every encryption. **The refusals are a courtesy; the protection is that plaintext never exists in the processor's process at all.**

- **Column packing** — an (N,d) array is d ciphertexts, one per column; linear scoring costs one multiplicative level → 8 ms for 200 rows
- **Key custody** — controller.key (secret, never travels) vs processor.ctx (evaluation keys only); every file carries the key-pair fingerprint, wrong key fails at open time
- **Dispatch** — numpy's official extension protocols (`__array_ufunc__` / `__array_function__`), the same mechanism CuPy and Dask use — not runtime hacks
- **Targeted hooks** — `pdpd.sklearn.install()` wraps only supported classes, type-gated: plain input untouched, encrypted input routed. Rule: **a hook may route or refuse, never compute**
- **Backend-neutral** — the .enc container records its scheme; CKKS today, TFHE-class next, selection at context creation

.ENC CONTAINER

```
CENC | v1 | header {  
  shape, packing,  
  scheme: "ckks",  
  columns, ctx_fp  
} | ciphertext blobs
```

BRANCHLESS LOGIC

```
gate = sigmoid(x)  
r = gate*b + (1-gate)*c  
# if/else, computed blind –  
# constant-time discipline
```

A7 · LEGAL MAPPING — HONEST VERSION

PROVISION	HOW PDPG RELATES
มาตรา 37(1) — security measures	FHE as a technical measure embedded in architecture — Privacy by Design, not bolted on
มาตรา 40 — processor duties & DPA	Contractual controls remain; PDPG converts the core promise ("don't look") into a technical impossibility
ประกาศฯ แจ้งเหตุละเมิด พ.ศ. 2565	Breach of ciphertext-only systems plausibly qualifies for the no-risk notification exemption (“มาตรการทางเทคโนโลยีที่เพียงพอ”) — interpretation to be confirmed with สคส.
Pseudonymization, not anonymization	Encrypted data remains personal data under PDPA/GDPR. We do not claim otherwise — PDPG reduces risk; it does not remove the law

A8 · THAI PDPA — PROVISION BY PROVISION

PROVISION	WHAT IT SAYS / HOW PDPG HELPS
มาตรา 37(1)	Controller must provide appropriate security measures — FHE is a strong, architecture-level technical measure (Privacy by Design, not an add-on)
มาตรา 37(4) + ประกาศฯ พ.ศ. 2565	Breach → notify สคส. within 72 h; the 2565 notification exempts breaches posing no risk , including data rendered unusable by adequate technological measures — a ciphertext-only breach plausibly qualifies (interpretation to confirm with สคส.)
มาตรา 40(1)(2) + วรรคสาม	Processor duties + mandatory DPA — contractual controls stay; PDPG turns the core promise (“don’t look at the data”) into a technical impossibility
มาตรา 83	Administrative fines up to ฿3M for controller-side failures — the stake that makes prevention cheaper than response
มาตรา 28 (cross-border)	Transfers abroad need adequate protection — shipping only ciphertext to a foreign processor is a strong supplementary safeguard

A9 · GDPR & INTERNATIONAL ALIGNMENT

INSTRUMENT	RELEVANCE
GDPR Art. 4(5)	Defines pseudonymisation — encrypted-with-a-key data is pseudonymized, still personal data. Same honest position we take under PDPA
GDPR Art. 25 & 32(1)(a)	Data protection by design; Art. 32 names encryption explicitly as a security measure
GDPR Art. 34(3)(a)	Breach communication to data subjects not required where encryption renders data unintelligible — the EU twin of the Thai 2565 exemption
Schrems II (CJEU C-311/18, 2020)	Invalidated Privacy Shield; put supplementary measures at the center of third-country transfers
EDPB Recommendations 01/2020	Lists strong encryption among effective supplementary measures for transfers — computing without decryption is the strongest version of that logic

Thai and EU regimes point the same way: encryption that survives **use** is the recognized risk-killer.

Credit scoring / IFRS-9

Bank → RegTech vendor computing ECL provisions. Today the vendor sees the loan book borrower-by-borrower; the bank only needed aggregates back. With PDPG the vendor's model runs on ciphertext. Demonstrated in our notebooks.

Government analytics

Agency → contracted analytics vendor: segmentation of citizen data computed blind; the segment decision is made key-side, by the agency. Demonstrated (KMeans, encrypted distances).

Next: AML screening

Watchlist checks where the vendor never learns who was asked about (no tipping-off) — the shape of Mastercard's cross-border FHE pilot; on our roadmap as TFHE-class backends land.

Fit today: linear scoring, aggregate statistics, segmentation — the bread and butter of controller→processor analytics.

WHO	WHAT
Apple (2024)	FHE (BFV) live in iOS — Live Caller ID Lookup, Enhanced Visual Search; open-sourced swift-homomorphic-encryption
Microsoft	Edge Password Monitor checks breached credentials over SEAL — the same library under PDPG; SEAL maintained by Microsoft Research
Google (2019)	Open-sourced Private Join and Compute — private set intersection for cross-org analytics
Mastercard (reported pilot)	Cross-border encrypted fraud checks across four jurisdictions — queries never persist abroad, no tipping-off
Zama	TFHE-rs / Concrete open source; ~US\$73M Series A (2024, reported) — serious capital behind the TFHE backends on our roadmap
DARPA DPRIVE	Dedicated FHE accelerator program targeting orders-of-magnitude speedups — the hardware end of the curve we inherit

A12 · WHY NOT ... — ALTERNATIVES COMPARED

APPROACH	TRADE-OFF	VERDICT
Anonymization	Destroys analytic utility; re-identification risk; hashed IDs remain indirect identifiers	often insufficient alone
Tokenization / masking	Reversible mapping exists somewhere; blocks computation on the values	good for display, not for compute
Differential privacy	Protects aggregate outputs with noise; vendor still needs raw data to compute	complementary, different layer
Federated learning	Data stays local but model updates leak; requires shipping the vendor’s code in	different topology
TEE / confidential computing	Trust moves to the CPU vendor; documented side-channel history (Foreshadow, Plundervolt)	complementary — hardware trust vs math
FHE (PDPG)	Compute cost; workload fit (linear today)	no trust in the processor at all

“Is encrypted data still personal data?”

Yes — pseudonymized, not anonymized; PDPA still applies and we say so everywhere. The value is risk elimination in use plus the notification-exemption path — not escaping the law.

“Why not a TEE / secure enclave?”

TEEs relocate trust to a CPU vendor and carry a side-channel record (Foreshadow, Plundervolt). FHE needs no trusted hardware — a leak is noise. The two compose.

“Why not just anonymize?”

True anonymization destroys utility and fails often (re-identification); hashes stay indirect identifiers. FHE keeps full computational utility with cryptographic protection.

“Isn’t FHE too slow?”

For today’s target workloads, measured: 8 ms to score 200 applicants. Heavy deep learning is backend-roadmap + hardware curve. We ship what fits and refuse what doesn’t — explicitly.

“What if the processor returns wrong results?”

FHE gives confidentiality, not verifiability — the threat model is honest-but-curious. Wrong-result risk stays contractual (exactly as today); ZK-style verifiability is research roadmap.

“What if the controller loses the key?”

controller.key **is** the data — standard key-management practice applies (KMS, backup, rotation). Keys never travel, so custody is a one-party problem.

“What about quantum computers?”

CKKS security rests on lattice problems (RLWE) — the same family NIST selected for post-quantum standards (FIPS 203). Better positioned than the RSA/ECC most infrastructure runs on.

“Can the controller steal the vendor’s model?”

Whoever decrypts scores can attempt model extraction — identical exposure to any scoring API today. A contract matter, and we state it plainly in our own fine print.

- **FHE** — Fully Homomorphic Encryption: math that lets you compute on encrypted data without decrypting it
- **Ciphertext** — the encrypted form; statistically indistinguishable from random noise
- **CKKS** — the FHE scheme for approximate real-number arithmetic (our current backend)
- **TFHE** — FHE “over the Torus”; enables exact comparisons and unlimited depth (roadmap)
- **Secret key** — decrypts; never leaves the data controller
- **Evaluation keys** — allow computation only; safe to ship to the processor
- **Multiplicative depth** — how many multiplications can chain before a refresh is needed
- **Pseudonymization vs anonymization** — reversible-with-a-key vs irreversible; encrypted data is the former, so data-protection law still applies

A16 · HONEST LIMITS — CURRENT BACKEND (CKKS)

- Arithmetic is **approximate** ($\sim 10^{-4}$) — fine for scoring and analytics, wrong tool for exact accounting
- **4 chained multiplications** per ciphertext; the 5th raises a clear error with alternatives — bootstrapping backends remove this limit
- Arrays $\leq 8,192$ rows; ciphertext ~ 1 MB per column; evaluation keys ~ 180 MB one-time
- Machine learning is **inference-only** (linear family) — training happens on plaintext at the model owner's side; trees & deep nets await next-gen backends
- FHE protects the **input data**, not the model: whoever decrypts scores can attempt model extraction — same exposure as any scoring API, handled by contract
- Not yet independently security-audited — stated plainly in our own docs

Roadmap

- Library axis: statsmodels, PyTorch (square-activation nets)
- Backend axis: TFHE (over the Torus) — exact comparisons, sort, unlimited depth; user code unchanged
- Scale axis: chunking beyond 8,192 rows, performance work

Asks

- **One pilot pair** — a real controller–processor deployment
- **Legal confirmation** — notification-exemption reading, with สคส. / counsel
- **Independent security review**
- R&D funding for the backend work above

- Regev — “On Lattices, Learning with Errors, Random Linear Codes, and Cryptography”, STOC 2005 (LWE)
- Gentry — “Fully Homomorphic Encryption Using Ideal Lattices”, STOC 2009 (FHE proven possible)
- Lyubashevsky, Peikert, Regev — “On Ideal Lattices and Learning with Errors over Rings”, EUROCRYPT 2010 (RLWE)
- Fan, Vercauteren — “Somewhat Practical Fully Homomorphic Encryption”, 2012 (BFV — the scheme Apple ships)
- Chillotti, Gama, Georgieva, Izabachène — “TFHE: Fast Fully Homomorphic Encryption over the Torus”, ASIACRYPT 2016 / J. Cryptology 2020
- Gilad-Bachrach et al. — “CryptoNets: Applying Neural Networks to Encrypted Data”, ICML 2016 (square-activation networks — our NN roadmap)
- Cheon, Kim, Kim, Song — “Homomorphic Encryption for Arithmetic of Approximate Numbers”, ASIACRYPT 2017 (CKKS — our current backend)
- Albrecht et al. — Homomorphic Encryption Security Standard, HomomorphicEncryption.org, 2018 (parameter security levels; our defaults follow it)
- Benaissa et al. — “TenSEAL: A Library for Encrypted Tensor Operations Using Homomorphic Encryption”, 2021
- NIST FIPS 203 — ML-KEM, 2024 (lattice-based post-quantum standardization; same hardness family as CKKS)

A19 · LINKS

- **Site / docs / playground** — pdpqlab.xyz
- **Source** — github.com/PDPG-lab/pydpdp (Thai README included)
- **Live demo** — Colab “getting started”, linked from the repo
- Repo docs: [/docs/design.md](#) (architecture) · [/docs/fine-print.md](#) (measurements, threat model, legal notes)
- License: PDPG Community License — [LICENSE.md](#) in the repository
- Report: [PDPG_PDPAHackathon2026.pdf](#) (submission document)



github.com/PDPG-lab/pydpdp

ฉบับภาษาไทย

สไลด์หลัก • ภาษาไทย

เนื้อหาเดียวกับสไลด์นำเสนอหลัก เรียบเรียงเป็นภาษาไทยสำหรับผู้อ่าน

PDPA HACKATHON 2026 · ทีม PDPG

เราคุ้มครองข้อมูล ขณะประมวลผล

Personal Data Protection Gateway — ผู้ประมวลผลข้อมูลคำนวณบนข้อมูลที่ตนอ่านไม่ได้
เข้ารหัสไป เข้ารหัสกลับ คุยเจออยู่กับผู้ควบคุมข้อมูลเท่านั้น

การเข้ารหัสทุกวันนี้คุ้มครองได้ 2 ใน 3 สถานะ

ขณะจัดเก็บ



เข้ารหัสบนดิสก์ — เป็นมาตรฐานอยู่แล้ว

ขณะส่งผ่าน



เข้ารหัสระหว่างทาง — เป็นมาตรฐานอยู่แล้ว

ขณะประมวลผล



ทันทีที่ต้องคำนวณ ข้อมูลต้องถูกถอดรหัส — ในระบบของคนอื่น

ข้อมูลอ่อนไหวออกจากบ้าน แต่ความรับผิดชอบไม่ได้ออกไปด้วย

ข้อมูลอ่อนไหวไปอยู่ที่ผู้ประมวลผล ในสภาพที่อ่านได้ขณะประมวลผล

พนักงาน · ผู้รับช่วง · เหตุรั่วไหล · การตั้งค่าผิด — ความเสี่ยงทั้งหมดของฝั่งผู้ประมวลผล กลายเป็นภาระของผู้ควบคุมข้อมูลโดยตรง

เมื่อเกิดเหตุ

- มาตรา 37(4) — แจ้ง สคส. ภายใน 72 ชั่วโมง
- มาตรา 83 — โทษปรับทางปกครองสูงสุด ฿3,000,000
- DPA (ม.40 ว.3) — สัญญาลงโทษได้ แต่ป้องกันไม่ได้

เบื้องหลังข้อมูลทุกแถวคือคนจริง — ข้อมูลที่รั่วกลายเป็นมิจาชีพ การสวมรอย และความเชื่อมั่นที่พังลง ทั้งต่อเจ้าของข้อมูลและชื่อเสียงขององค์กร **ค่าปรับเรียกคืนสิ่งเหล่านี้ไม่ได้**

จะเป็นอย่างไร ถ้าผู้ประมวลผล ไม่เคยเห็นข้อมูลเลยตั้งแต่แรก

PDPG: อัลกอริทึมเดิม โค้ดเดิม — รันบน ciphertext

จัดเก็บ ✓ · ส่งผ่าน ✓ · และตอนนี้ ประมวลผล ✓

เว็บไซต์ เอกสาร และสนามทดลอง (playground)

- แหล่งเรียนรู้สำหรับผู้ที่ไม่ใช่โปรแกรมเมอร์ — DPO ฝ่ายกฎหมาย ผู้บริหาร
- Playground พาเดินตั้งแต่ตัวเลขที่ถูกฝึก จนถึง โมเดล ML ที่คำนวณโดยไม่เห็นข้อมูล
- จากแนวคิด → คลิกเดียวถึงการเข้ารหัสจริงบน Colab
- มีทั้งภาษาไทยและอังกฤษ

ติดตั้ง 1 บรรทัด · ทดสอบ 111 รายการ · notebook 5 ชุด

- กรรมการเปิด Colab แล้วกด Run all ได้ระหว่างการตัดสินใจ — เข้ารหัสจริง คำนวณจริง ถอดรหัสจริง ~2 นาที
- แก่โค้ด ทดลองต่อ และใช้เป็นโครงเริ่มต้นของงานจริงได้ทันที
- รองรับ numpy · pandas · scikit-learn (บางส่วน โดยขอบเขตคือสิ่งที่แบ็กเอนด์ทำได้จริง)
- github.com/PDPG-lab/pydpdg — พร้อม README ภาษาไทย

ฝ่ายละ 2 บรรทัด ที่เหลือเป็นหน้าที่ของเรา

ผู้ควบคุมข้อมูล

```
ctx = pdpg.Context.create()
pdpg.encrypt(df, ctx).save("data.enc")
# ผู้เดียวที่ถอดรหัสผลได้
result = pdpg.load("result.enc").decrypt()
```

ผู้ประมวลผลข้อมูล

```
pdpg.activate("processor.ctx") # ไม่มีกุญแจลับ
pdpg.install()
X = np.load("data.enc")
scores = X @ w + b # โค้ดเดิม มองไม่เห็นข้อมูล
```

8 ms

ให้คะแนน 200 ราย บนข้อมูลเข้ารหัส

2.5×10^{-4}

คลาดเคลื่อนสูงสุดเทียบ plaintext

111

ชุดทดสอบอัตโนมัติ ผ่านทั้งหมด

ทำไมต้องตอนนี้

คณิตศาสตร์อายุ 15 ปี — มาถึงมือถือของทุกคนเมื่อปีที่แล้ว

- 2009 — Gentry พิสูจน์ว่า FHE เป็นไปได้
- 2017 — CKKS: คำนวณเลขจริงโดยประมาณได้ (สคิมที่เราใช้)
- 2024 — Apple ใช้ FHE จริงใน iOS · Microsoft SEAL รันใน Edge
- ถัดไป — แบ็กเอนด์ตระกูล TFHE (เปรียบเทียบ/จัดเรียง/ความลึกไม่จำกัด) และฮาร์ดแวร์เฉพาะทาง

PDPG ออกแบบให้สลับแบ็กเอนด์ได้ — งานที่เขียนวันนี้ได้ประโยชน์จากพัฒนาการทั้งหมดโดยอัตโนมัติ
เอนจินเก่งขึ้น โค้ดคุณไม่ต้องแก้

ฟรีเพื่อประโยชน์สาธารณะ — ผู้ได้ประโยชน์ที่สุดเป็นผู้สนับสนุน

PDPG Community · ฟรี

- ภาครัฐและหน่วยงานสาธารณะ
- การศึกษา งานวิจัย องค์กรไม่แสวงกำไร
- บริษัทรายได้ต่ำกว่า ฿50 ล้าน (ไทย) / \$1M

Enterprise · มีสัญญาอนุญาต

- องค์กรขนาดใหญ่ใช้ stack เดียวกัน (source-available)
- มีบริการ support และ consulting
- ค่าสัญญาอนุญาตหล่อเลี้ยงส่วนที่ฟรี

ผู้ประมวลผลไม่มีวันเห็นข้อมูล นั่นแหละคือผลิตภัณฑ์

pdpqlab.xyz · github.com/PDPG-lab/pydpdpq